## 1. CONFORMACIÓN DEL EQUIPO SCRUM Y VALIDACIÓN DE ROLES

| Miembro | Rol |
|---|---|
| Matias | Product Owner (PO) |
| Alison | Scrum Master (SM) |
| Benja, Franco, Bianca, Leandro | Developer |

## 2. SPRINT PLANNING: PARÁMETROS DEL SPRINT E HISTORIAS

| Parámetros | Definición para Sprint #1 |
|---|---|
| Sprint Goal | Desarrollar la visualización de un calendario semanal y su estructura básica de disponibilidad. |
| Time-box | 8 días |
| Capacidad / SP | 8 SP |
| Sprint Backlog | *Las tareas que van a realizar para cumplir el objetivo:* clases y servicios, endpoint API, frontend React, CSS y pruebas. |
| Tablero Kanban | POR HACER (To Do) \| EN CURSO (In Prog) \| EN REVISIÓN \| FINALIZADO (Done) |

### 2.1 Historia de Usuario:

ID y Título: US-9 Visualizar Calendario de Disponibilidad

Épica asociada: Gestión y visualización de horarios y disponibilidad.

Estructura Card (3C): Como usuario, quiero visualizar el calendario semanal de clases, para conocer los horarios, la disponibilidad de cupos y organizar mi rutina.

Criterios de aceptación:
Escenario 1: Visualización general de la grilla
Dado que el usuario accede al módulo de horarios, cuando carga la página, entonces se debe mostrar un calendario estructurado por días de la semana.

Escenario 2: Datos de las clases
Dado que el usuario visualiza el calendario, entonces cada bloque de clase debe mostrar el nombre de la clase y su franja horaria.

Escenario 3: Indicadores de disponibilidad
Dado que existen clases con cupos limitados, entonces el sistema debe diferenciar visualmente (ej. con etiquetas o colores) cuáles clases tienen disponibilidad y cuáles están completas (*sin cupos*).

Escenario 4: Información de la clase (opcional)
Dado que el usuario visualiza las clases y su horario, debe poder acceder de alguna forma a la información del profesor que dicta dicha clase.

**2.2 Desglose técnico en Sub-tasks:**

| ID | Descripción | Responsable | Est. Hrs. |
|---|---|---|---|
| US-29 | Cree el diagrama UML teniendo en cuenta la interacción de las diferentes entidades en el contexto de la funcionalidad a realizar. | Benja | 3 |
| US-36 | Cree una rama que la llamamos “develop” para poder pushear el incremento. | Benja | 1 |
| US-30 | Implementar el controller encargado de gestionar las solicitudes relacionadas con la funcionalidad. | Bianca | 1 |
| US-31 | Implementar las clases del modelo y sus relaciones según el diseño definido en el UML. | Bianca | 1 |
| US-44 | Verificar mediante pruebas el funcionamiento del controller y sus principales casos de uso. | Bianca | 1 |
| US-40 | Implementar los servicios encargados de gestionar las operaciones necesarias del modelo. | Leandro | 1 |
| US-42 | Verificar mediante pruebas el correcto funcionamiento de los servicios implementados. | Leandro | 1 |
| US-32 | – | – | – |
| US-33 | – | – | – |
| US-43 | Creé el proyecto con Vite para tener un servidor local y el entorno de desarrollo. | Franco | 1 |
| US-34 | Utilice el routing que me brinda React para realizarlo, para que el /clases responda con el componente correspondiente. | Franco | 1 |
| US-35 | Lo implementé con React y TypeScript el componente que realiza la tabla para su visualización en el /clases. | Franco | 1 |
| US-37 | Cree clases estáticas que no están cargadas en el backend para poder ver cómo se renderiza el componente. | Benja | 1 |
| US-39 | Modifiqué el componente para poder realizar el CSS del mismo y lograr el estilo deseado. | Benja | |
| US-41 | – | – | – |

### 2.3 D.O.D.: Definición de hecho:

Se considera que la historia de usuario está terminada cuando se cumplen las siguientes condiciones:

- La visualización cumple con lo esperado y respeta los criterios de aceptación definidos.

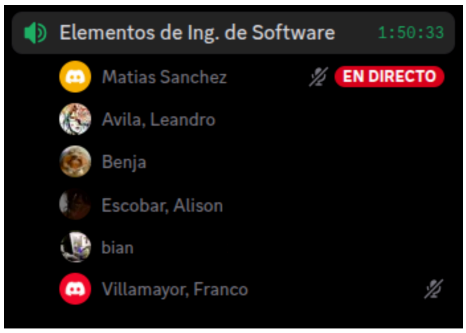
- El frontend y el backend están implementados y permiten realizar el flujo previsto para la funcionalidad.

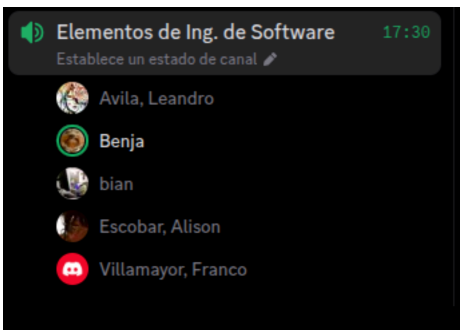
- Se tienen hechas las pruebas mínimas de los casos exitosos y de algunos casos de error relevantes.

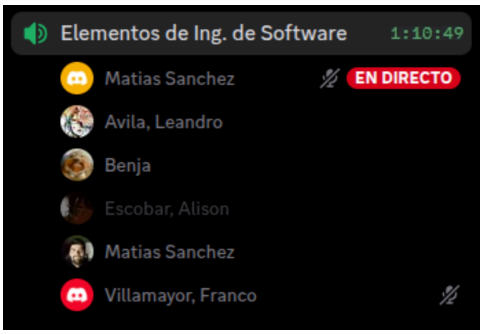
## 3. BITÁCORA DE REFINAMIENTO Y TÉCNICAS DE VERTICAL SLICING

| *Sprint* | *Incidencia* | *Acción* | *Detalle y justificación del refinamiento* |
|---|---|---|---|
| *#1* | *Creación de historia de usuario: Clase* | *Se agrega US al product backlog.* | *La US de creación de clases requiere de la existencia de la clase. Lo que llevó a una división de la US aplicando vertical slicing.* |
| *#1* | *Creación de historia de usuario: Registro de cupos* | *Se agrega US al product backlog.* | *No se especificaba qué ocurría con los cupos. Lo que llevó a una división de la US aplicando vertical slicing.* |
| *#1* | *Refinamiento de historia de usuario: Agregar clase* | *Se modifica US en Product Backlog.* | *Ambigüedades que requieren análisis y división en más US.* |
| *#1* | *Refinamiento de historia de usuario: Eliminar clase* | *Se modifica US en Product Backlog.* | *Un análisis de refinamiento llevó a su modificación y posterior creación de US.* |

## 4. DAILY SCRUM




03/10: Se convocó a una reunión con el PO, SM y el equipo de desarrollo para definir qué user story se trabajaría durante el sprint.

- Se seleccionó la US-9 — Visualización del calendario.
- Se definió el objetivo del sprint.
- Las tareas técnicas se encuentran pendientes de definición.
- Sin impedimentos.

04/10:
- Se intercambiaron mensajes por WhatsApp, pero no se logró coordinar una reunión.
- Sin acuerdos ni impedimentos identificados.

05/10:
- Se conversó por WhatsApp sobre una posible división de las tareas.
- La distribución quedó pendiente de definición.

06/10:
- Se realizó una reunión entre el SM y los developers.
- Se definieron y distribuyeron las subtareas del sprint.

07/10 y 08/10:
- Los developers avanzaron con la implementación de las tareas asignadas.

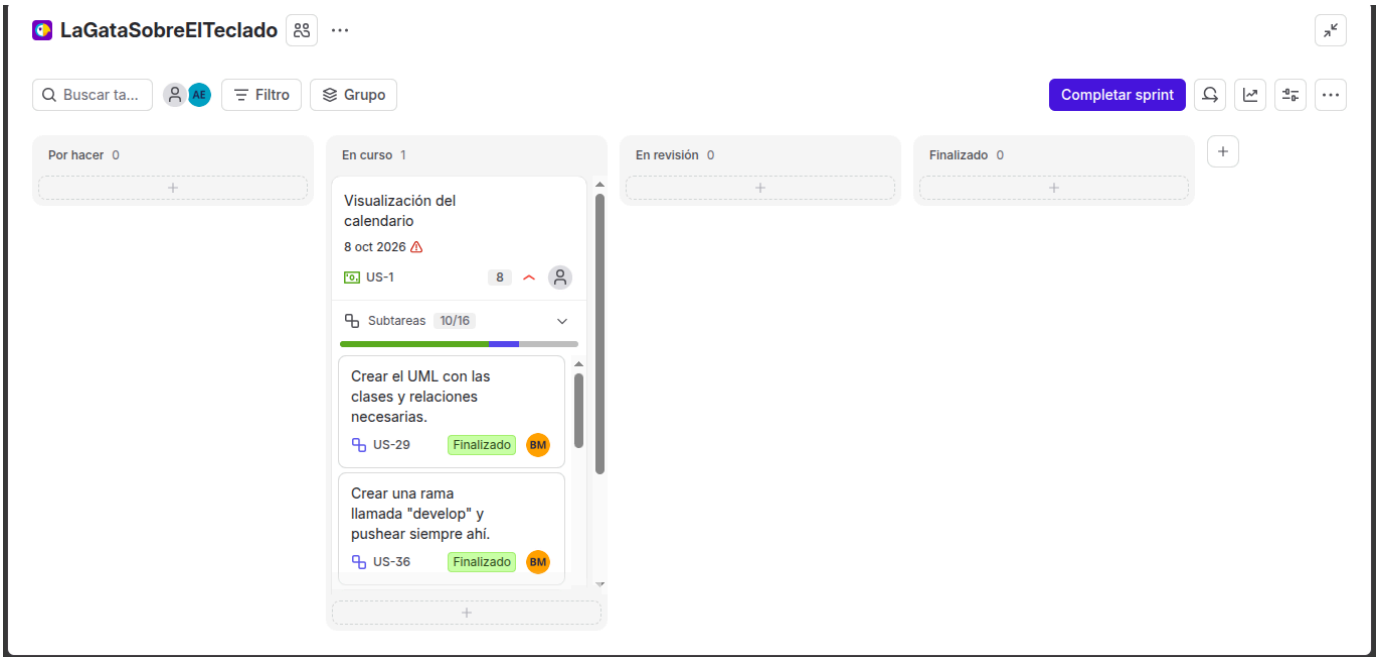
09/10:
- Se realizó la Sprint Review y la retrospectiva con todo el equipo.
- Se probaron los avances y se acordó dar por finalizado el sprint.
- Se identificaron aciertos y dificultades, y se asignaron los roles para el próximo sprint.
- Sin impedimentos relevantes.

## 5. SPRINT REVIEW (DOBLE INSTANCIA: GESTIÓN Y DEMO ACTIVA)

Durante la Sprint Review se decidió considerar terminada la historia de usuario, ya que se completaron las subtareas consideradas esenciales para cumplir con el objetivo del sprint. Las tareas opcionales no se implementaron, dado que no eran indispensables para alcanzar la funcionalidad principal.

La demostración permitió observar la visualización del calendario semanal y comprobar el avance realizado en la integración de los componentes necesarios. De esta manera, se pudo evaluar el resultado obtenido en relación con los criterios de aceptación definidos para la historia de usuario.

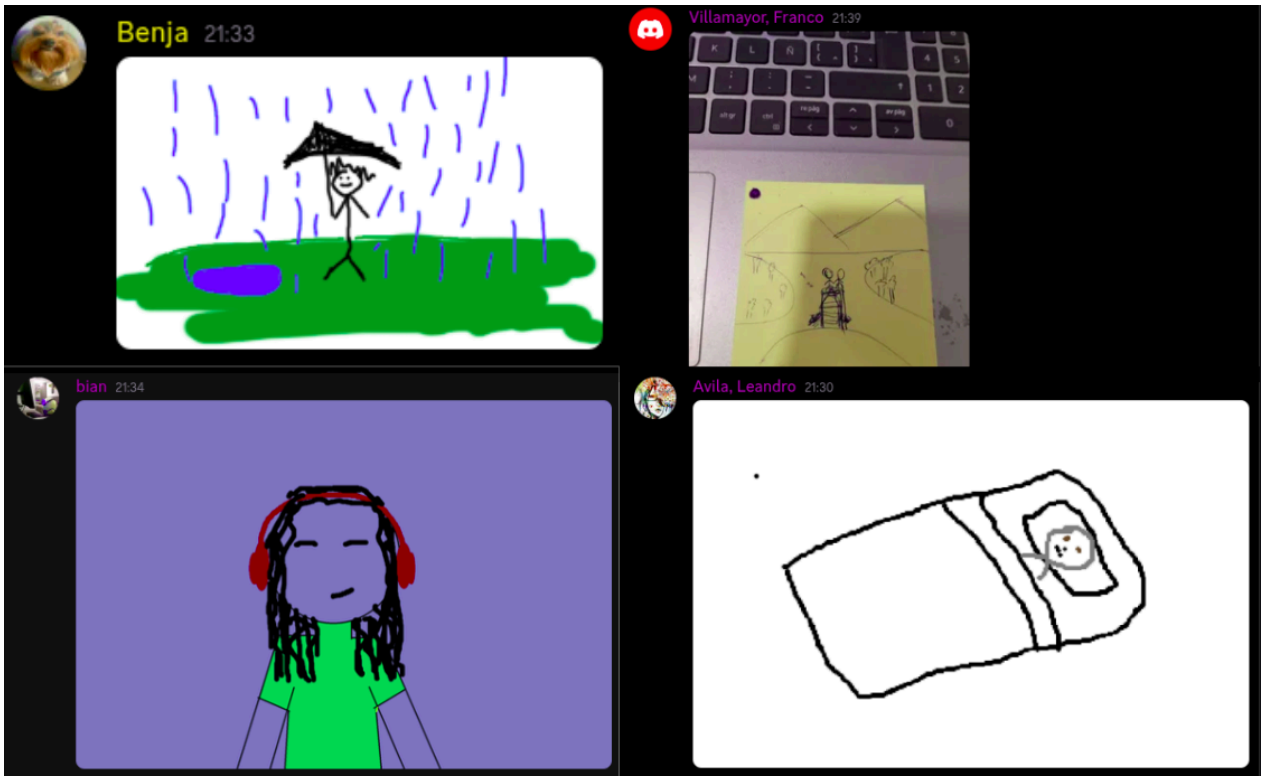
ANTES DE CERRAR EL SPRINT:



## 6. SPRINT RETROSPECTIVE

### 6.1 Primera fase: dinámica “Dibújate cómo podrías relajarte”.

*Es una actividad de precalentamiento donde cada integrante dibujó su forma ideal de distensión. Favorece la seguridad y distiende el estrés antes de la introspección.*




### 6.2 Recolección de datos:

Aciertos

Reunión inicial para distribuir las subtareas: Permitió que todos los integrantes tuvieran una visión general de cómo comenzar a trabajar y qué actividades debían realizar.

Simplicidad de las tareas iniciales: Las tareas planteadas fueron relativamente sencillas, lo que facilitó su comprensión y ejecución dentro del tiempo disponible.

Separación entre backend y frontend: Permitió distribuir las responsabilidades y avanzar en paralelo sobre las distintas partes de la aplicación.

Visualización de la funcionalidad: Contar con una representación gráfica del calendario facilitó la interpretación de los criterios de aceptación y permitió observar de manera concreta el resultado esperado.

Dificultades

Algunas subtareas se definieron desde una perspectiva técnica y no como incrementos pequeños de funcionalidad. Esto podría generar dependencias entre integrantes y dificultar la integración de los componentes.

No se llegó a comprender completamente el dominio del problema antes de dividir las tareas. Esto dificultó identificar todas las relaciones entre las entidades y definir con precisión las responsabilidades de cada componente.

Algunas actividades requerían que otras estuvieran terminadas previamente, lo que podía limitar el trabajo en paralelo y generar esperas durante el desarrollo.

### 6.3 Indagar causas:

Al priorizar la distribución inicial del trabajo, no se analizaron suficientemente las dependencias entre las subtareas ni se evaluó si cada una aportaba un incremento funcional verificable de manera independiente. Esto dificultó aplicar de forma consistente el enfoque de *vertical slicing*.

Además, la separación entre backend y frontend facilitó la distribución del trabajo.

### 6.4 Decidir qué hacer:

Para mejorar la organización y el desarrollo de los próximos sprints, se proponen las siguientes acciones:

Dedicar más tiempo al inicio del sprint a comprender las entidades, sus relaciones y las reglas principales del sistema.

Procurar que la división de las subtareas sea pequeña, clara y verificable, priorizando incrementos funcionales que puedan probarse de manera independiente.

Analizar qué tareas requieren otras actividades previas y distribuirlas teniendo en cuenta la disponibilidad de cada integrante y los conocimientos necesarios.